SETUP-LINUX

Local / single-host, with script + manual + GUI options for every step

2026-08-28

- OpenHIE Stack — Linux Setup & Bring-up
  - 0. Prerequisites
  - 1. Start the containers
  - 2. Trust the OpenHIM self-signed certificate
  - 3. OpenHIM first login and root password
  - 4. Keycloak realm
  - 5. Realm public key into OpenHIM
  - 6. Confirm channels (automatic — no action needed)
  - 7. Onboard a hospital
  - 8. Smoke test — end to end
  - 9. FHIR Intake mediator (`POST /ingest`)
  - 10. Duplicate-review mediator (`/admin/review/*`)
  - 11. Service management & reboot persistence
  - 12. Starting over (full reset) — destructive
  - 13. Quick reference
  - 14. Troubleshooting

OpenHIE Stack — Linux Setup & Bring-up

Local / single-host bring-up on a Linux workstation or server. This is the Linux edition of `SETUP-WINDOWS.md`. For a public VPS with a domain and HTTPS, see `VPS-DEPLOYMENT.md`.

Every step that has an automation script is given **three ways to do it**:

- **Option A — Script**: run the bundled script.
- **Option B — Manual (CLI)**: the exact commands the script runs.
- **Option C — GUI**: the same operation through a web console, where one exists.

They are interchangeable — a client created by the script, by hand, or in the GUI is identical.

`<like-this>` = placeholder. Run everything from the repo root unless stated.

---

0. Prerequisites

0a. Packages

Tool	Why	Debian/Ubuntu install
Docker Engine + Compose v2	runs the whole stack	`curl -fsSL https://get.docker.com | sh`
Git	clone the repo	`sudo apt-get install -y git`
`jq`	`keycloak/*.sh`, `extract-realm-pubkey.sh`	`sudo apt-get install -y jq`

Tool	Why	Debian/Ubuntu install
`curl`	scripts + health checks	usually present
`openssl`	`create-jwt-client.sh`, `get-jwt-client-token.sh`	usually present
`python3`	`onboard-hospital.sh` reads the JWT `sub` claim	`sudo apt-get install -y python3`
`wireguard` (`wg`, `wg-quick`)	only if you tunnel to JeMPI (0e Option B)	`sudo apt-get install -y wireguard`
`fold`, `base64`	`extract-realm-pubkey.sh`, `create-jwt-client.sh`	part of `coreutils` (present)

One line for Debian/Ubuntu:

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl git jq openssl python3 wireguard ufw
curl -fsSL https://get.docker.com | sh
sudo systemctl enable --now docker
```

0b. Run Docker without sudo

```
sudo usermod -aG docker "$USER"
newgrp docker                 # or log out and back in
docker run --rm hello-world   # must print "Hello from Docker!"
```

Skip this and every `docker compose` below needs `sudo`.

Verify the toolchain:

```
docker compose version        # v2.x
git --version
jq --version && openssl version && python3 --version
```

0c. Get the code

```
git clone https://github.com/ETech-Nepal/openhim.git openhie
cd openhie
```

Keep the path free of spaces and off any network / cloud-synced mount — Docker bind-mounts several paths from this tree (`openhim-config`, `hapi-config`, `jempi-config`).

0d. Configuration files (none are in git)

Four mediator `.env` files and one placeholder key file:

```
for d in mediator admin-mediator patient-portal-mediator fhir-intake-mediator; do
  cp "$d/.env.example" "$d/.env"
done
mkdir -p openhim-config
[ -f openhim-config/realm-public.pem ] || touch openhim-config/realm-public.pem
chmod 600 */.env openhim-config/realm-public.pem
```

Why the empty `realm-public.pem`: `docker-compose.yml` bind-mounts it into `openhim-core`. If it does not exist when the stack first starts, Docker creates the mount target as a **directory**, the entrypoint's `cat` yields nothing, `authentication_jwt_secretOrPublicKey` ends up empty, and every token fails with `JWT could not be verified: secret or public key must be provided`. Step 6 overwrites this file with the real key.

The defaults in the examples are correct for a local Docker run. Only these need your input:

File	Variable	Set to
all four	`OPENHIM_PASSWORD`	the OpenHIM root password from step 3 — **blank in the examples**; mediators crash-loop until it is filled
`mediator/.env`, `admin-mediator/.env`	`JEMPI_BASE_URL`	see step 0e
`patient-portal-mediator/.env`	`PORTAL_PATIENT_SHARED_SECRET`	only if you run the patient-portal BFF — must equal its `SESSION_JWT_SECRET`
`mediator/.env`	`FACILITY_REGISTRY_JSON`	only as the step-7 fallback

Do **not** create a root `.env` for a local run — every variable has a `localhost` default in `docker-compose.yml`, and a half-filled root `.env` points Keycloak and OpenHIM at an issuer your tokens do not carry. The root `.env` is a VPS-only file (`VPS-DEPLOYMENT.md`).

0e. Decide how the mediators reach JeMPI

JeMPI is **not** in this compose stack. It runs on a separate host (`202.51.71.81`). Both `mediator/.env` and `admin-mediator/.env` need `JEMPI_BASE_URL`. Pick one now; you can change it later with `docker compose up -d --force-recreate mediator admin-mediator`.

Option A — public port (simplest, test data only):

```
sed -i 's#^JEMPI_BASE_URL=.*#JEMPI_BASE_URL=http://202.51.71.81:50000/JeMPI#' \
  mediator/.env admin-mediator/.env
curl http://202.51.71.81:50000/JeMPI/countRecords     # must return JSON
```

Traffic crosses the internet unencrypted and JeMPI has no TLS/auth of its own — restrict `:50000` to your IP on the JeMPI host firewall.

Option B — WireGuard tunnel (keeps the default `10.8.0.1`): covered in `VPS-DEPLOYMENT.md` §5. On a workstation:

```
sudo cp <your-wg0.conf> /etc/wireguard/wg0.conf     # AllowedIPs must include 10.8.0.0/24
sudo wg-quick up wg0
curl http://10.8.0.1:50000/JeMPI/countRecords       # from host
```

If the host reaches JeMPI but a container hangs:

```
sudo iptables -t nat -A POSTROUTING -s 172.16.0.0/12 -o wg0 -j MASQUERADE
sudo sysctl -w net.ipv4.ip_forward=1
```

---

1. Start the containers

```
docker compose up -d
docker compose ps
```

Expect running: `hapi-db`, `mongo-db`, `openhim-core`, `openhim-console`, `fhir`, `keycloak`, `mediator`, `admin-mediator`, `patient-portal-mediator`, `fhir-intake-mediator`.

- `caddy` / `caddy-ip` are **not** started locally — they sit behind Compose profiles (`vps` / `vps-ip`) used only on a VPS.
- `fhir` can show `unhealthy` for the first ~30–60 s while HAPI boots — normal.
- The mediators **crash-loop** until steps 3 and 6 are done — expected.

> **Never run `docker compose down`.** It can silently wipe Keycloak's embedded H2 realm (Keycloak boots a fresh empty DB with no error). Use `docker compose stop` / `docker compose start`. `docker compose restart keycloak` and `up -d --force-recreate keycloak` are safe. `down -v --rmi all` is only for a deliberate full reset.

First run pulls several GB of images. If a container exits immediately, read its log first: `docker compose logs --tail 50 <service>`.

---

2. Trust the OpenHIM self-signed certificate

OpenHIM Core's API uses a self-signed cert. Browser calls from the Console fail silently until it is trusted.

- **GUI:** open https://localhost:8080/heartbeat, click **Advanced → Proceed**. You should see a small JSON heartbeat body.
- **CLI:** add `-k` — `curl -k https://localhost:8080/heartbeat`.

---

3. OpenHIM first login and root password

There is no CLI for the first-login password change — it is a GUI step, then the value is propagated to the mediators by script or by hand.

3a. Set the password (GUI)

1. Open http://localhost:9000 (OpenHIM Console).
2. Log in `root@openhim.org` / `openhim-password`.
3. You are prompted to set a new password (or **Users → your user → Edit**). **Record it.**

3b. Propagate to the four mediators

Option A — script-style one-liner:

```
NEWPW='<the-password-you-set>'
for d in mediator admin-mediator patient-portal-mediator fhir-intake-mediator; do
  sed -i "s/^OPENHIM_PASSWORD=.*/OPENHIM_PASSWORD=${NEWPW}/" "$d/.env"
done
docker compose up -d --force-recreate mediator admin-mediator patient-portal-mediator fhir-
        intake-mediator
```

Option B — manual: edit `OPENHIM_PASSWORD=` in each of the four `.env` files in a text editor, then run the `docker compose up -d --force-recreate ...` line above.

> `docker compose restart` does **not** reload `.env`. Always use `up -d --force-recreate` after an `.env` change.

Verify: OpenHIM Console → **Mediators** → all four show green / heartbeating.

---

4. Keycloak realm

Keycloak boots with `--import-realm`, which auto-creates the `openhie` realm plus a baseline `demo-client` / `demo` / `demo123` from `keycloak/realm-export.json` **if the realm is missing**. Check first:

```
curl -fsS http://localhost:8083/realms/openhie/.well-known/openid-configuration | jq .issuer
```

A non-empty issuer URL → the realm exists, skip to step 5. Otherwise create it:

Option A — script

```
./keycloak/setup.sh
```

Creates the realm, `demo-client` (public, direct-access-grant), user `demo`/`demo123` with the profile fields Keycloak's User Profile requires, and prints the demo user's `sub`.

Option B — manual (CLI)

```
KC="docker exec keycloak /opt/keycloak/bin/kcadm.sh"
$KC config credentials --server http://localhost:8080 --realm master --user admin --password
        admin
$KC get realms/openhie >/dev/null 2>&1 || $KC create realms -s realm=openhie -s enabled=true

# demo client + user (optional — only for password-grant demo tokens)
$KC create clients -r openhie -s clientId=demo-client -s publicClient=true \
  -s standardFlowEnabled=false -s directAccessGrantsEnabled=true
$KC create users -r openhie -s username=demo -s enabled=true
$KC set-password -r openhie --username demo --new-password demo123
SUB=$($KC get users -r openhie -q username=demo --fields id --format csv --noquotes)
$KC update users/$SUB -r openhie -s 'requiredActions=[]' -s emailVerified=true \
  -s email=demo@example.com -s firstName=Demo -s lastName=User
```

Option C — GUI

1. Keycloak Admin Console http://localhost:8083, log in `admin` / `admin`.
2. Top-left realm dropdown → **Create realm** → **Realm name** `openhie` → **Create**.

3. (Optional demo client) **Clients → Create client** → Client ID `demo-client`, Client authentication **Off**, **Direct access grants** on, others off → Save.
4. (Optional demo user) **Users → Add user** → username `demo`, email `demo@example.com`, first/last name, **Email verified** on, no required actions → Create → **Credentials → Set password** `demo123`, **Temporary** off.

Verify (any option):

```
curl -fsS http://localhost:8083/realms/openhie/.well-known/openid-configuration | jq .issuer
# → "http://localhost:8083/realms/openhie"
```

---

5. Realm public key into OpenHIM

OpenHIM verifies every JWT against the realm's RS256 **public key read from a file at boot** (`openhim-config/realm-public.pem`) — not a live call to Keycloak. Any time the realm is recreated its keypair rotates and this file must be refreshed.

Option A — script

```
./keycloak/extract-realm-pubkey.sh
docker compose up -d --force-recreate openhim-core
docker compose restart mediator admin-mediator patient-portal-mediator fhir-intake-mediator
```

Option B — manual (CLI)

```
KEY=$(curl -fsS http://localhost:8083/realms/openhie | jq -r .public_key)
{ echo "-----BEGIN PUBLIC KEY-----"; echo "$KEY" | fold -w64; echo "-----END PUBLIC
        KEY-----"; } \
  > openhim-config/realm-public.pem
openssl pkey -pubin -in openhim-config/realm-public.pem -noout && echo OK

docker compose up -d --force-recreate openhim-core
docker compose restart mediator admin-mediator patient-portal-mediator fhir-intake-mediator
```

Option C — GUI (to read the key; the file still has to be written)

Keycloak Admin Console → realm `openhie` → **Realm settings → Keys** → the **RS256** row → **Public key** → copy. Paste it between `-----BEGIN PUBLIC KEY-----` / `-----END PUBLIC KEY-----` lines (wrapped at 64 chars) into `openhim-config/realm-public.pem`, then run the two `docker compose` commands from Option A.

> **If `realm-public.pem` is a directory** (Docker created the mount target before the file existed): `docker compose stop openhim-core && rm -rf openhim-config/realm-public.pem` then redo this step.

The OpenHIM JWT settings are already in `docker-compose.yml`: `authentication_jwt_algorithms: "RS256"` (a plain string, **not** `["RS256"]`), `authentication_jwt_audience: "account"`, `authentication_jwt_issuer: http://localhost:8083/realms/openhie`.

Transaction-port error codes:

Code	Meaning
401	token missing / bad signature / `iss` or `aud` mismatch
403	Client mapped but lacks a channel role
404	no channel matched the URL

---

6. Confirm channels (automatic — no action needed)

Each mediator's `src/openhim.js` calls `registerMediator()` then `syncChannels()` on every boot, **upserting** its channel(s) via the OpenHIM API (OpenHIM only materialises `defaultChannelConfig` on the first registration of a URN). So there is no manual "add the channel" step.

Verify (GUI): OpenHIM Console → **Channels** shows all four, **Enabled**:

Channel	URL pattern	Methods	Role
NP FHIR Orchestrator	`^/fhir(/.*)?$`	GET POST PUT DELETE	`np-fhir-clients`
NP FHIR Intake	`^/ingest/?$`	POST	`np-fhir-intake-clients`
NP Patient Admin Review	`^/admin/.*$`	GET POST	`np-admin-clients`
NP Patient Portal	`^/patient-portal/.*$`	GET	`patient-portal-clients`

If one is missing/stale: `docker compose restart <mediator>` re-upserts it.

The `admin-mediator` also creates its own Postgres database `admin_review` on the `hapi-db` container at boot (`src/db/pool.js` `migrate()`). Verify: `docker exec hapi-db psql -U admin -d admin_review -c '\dt'` lists `review_tasks`.

---

7. Onboard a hospital

A hospital needs, all keyed on its **NHFR code** (Nepal Health Facility Registry code): a HAPI `Organization`, an RSA keypair, a Keycloak `private_key_jwt` client whose token `sub` is forced to the NHFR code, and an OpenHIM `Client` with role `np-fhir-clients`.

Option A — script (one command)

```
./keycloak/onboard-hospital.sh 3130400221 "Bharatpur Hospital"
```

Internally: `create-org.sh` → `create-jwt-client.sh` → add a `oidc-hardcoded-claim-mapper` forcing `sub` → verify the token `sub` (fails at step 4/5 if Keycloak rejects the override) → `create-hip-client.js`. Idempotent — safe to re-run.

The private key lands at `keycloak/keys/3130400221.key.pem` (gitignored) — hand it to the hospital over a secure channel, then delete your copy.

Option B — manual (CLI)

```
NHFR=3130400221 ; NAME="Bharatpur Hospital"
NHFR_SYS=http://mohp.gov.np/fhir/NamingSystem/nhfr
KC="docker exec keycloak /opt/keycloak/bin/kcadm.sh"

# 7b.1 — HAPI Organization (conditional update, one per NHFR code)
curl -sS -k -X PUT "http://localhost:8081/fhir/Organization?identifier=${NHFR_SYS}|${NHFR}" \
  -H 'Content-Type: application/fhir+json' \
  -d "{\"resourceType\":\"Organization\",\"active\":true,\"name\":\"${NAME}\",
       \"identifier\":[{\"use\":\"official\",
         \"type\":{\"coding\":[{\"system\":\"http://terminology.hl7.org/CodeSystem/v2-0203\",
        \"code\":\"FI\"}]},
         \"system\":\"${NHFR_SYS}\",\"value\":\"${NHFR}\"}]}" | jq .

# 7b.2 — RSA keypair + Keycloak private_key_jwt client
mkdir -p keycloak/keys
openssl genrsa -out "keycloak/keys/${NHFR}.key.pem" 2048
openssl rsa -in "keycloak/keys/${NHFR}.key.pem" -pubout -out "keycloak/keys/${NHFR}.pub.pem"
PUB_DER_B64=$(openssl rsa -in "keycloak/keys/${NHFR}.key.pem" -pubout -outform DER 2>/dev/null
        | base64 -w0)
$KC config credentials --server http://localhost:8080 --realm master --user admin --password
        admin
$KC create clients -r openhie -s clientId=$NHFR -s publicClient=false \
  -s standardFlowEnabled=false -s serviceAccountsEnabled=true \
  -s clientAuthenticatorType=client-jwt \
  -s "attributes.\"jwt.credential.public.key\"=$PUB_DER_B64" \
  -s 'attributes."token.endpoint.auth.signing.alg"=RS256' \
  -s 'attributes."use.jwks.url"=false'
CLIENT_UUID=$($KC get clients -r openhie -q clientId=$NHFR --fields id --format csv --noquotes)

# 7b.3 — force the token sub to the NHFR code
$KC create "clients/$CLIENT_UUID/protocol-mappers/models" -r openhie \
  -s name=sub-as-nhfr -s protocol=openid-connect \
  -s protocolMapper=oidc-hardcoded-claim-mapper \
  -s 'config."claim.name"=sub' -s "config.\"claim.value\"=$NHFR" \
  -s 'config."jsonType.label"=String' -s 'config."access.token.claim"=true' \
  -s 'config."id.token.claim"=false' -s 'config."userinfo.token.claim"=false'

# 7b.4 — verify
TOKEN=$(./keycloak/get-jwt-client-token.sh $NHFR)
python3 -c 'import base64,json,sys;p=sys.stdin.read().strip().split(".")[1];p+="="*(-len(p)
        %4);print(json.loads(base64.urlsafe_b64decode(p))["sub"])' <<<"$TOKEN"   # must print
        3130400221

# 7b.5 — OpenHIM Client (clientID = the sub, role np-fhir-clients, no password)
docker exec -e HIP_CLIENT_SUB="$NHFR" -e HIP_CLIENT_NAME="$NAME" \
  mediator node scripts/create-hip-client.js
```

Option C — GUI

1. **HAPI Tester** http://localhost:8081/ → **Organization** → **Create** → paste the Organization JSON from 7b.1 → **Create/Update**. (Or **Search** by `identifier` = `http://mohp.gov.np/fhir/NamingSystem/nhfr|3130400221` to confirm exactly one.)
2. **Keycloak Admin Console** → realm `openhie` → **Clients** → **Create client** → Client ID `3130400221`, **Client authentication** On, enable **Service account roles**, disable **Standard flow** → Save.
   - **Credentials** tab → **Client Authenticator** = Signed Jwt, **Signing algorithm** `RS256`.
   - **Keys** tab → **Use JWKS URL** Off → **Import Key** → paste the contents of `keycloak/keys/3130400221.pub.pem` (generate it with the two `openssl` commands from 7b.2).

    - **Client scopes** → `3130400221-dedicated` → **Add mapper** → **By configuration** → **Hardcoded claim** → Name `sub-as-nhfr`, **Token Claim Name** `sub`, **Claim value** `3130400221`, **Claim JSON Type** String, **Add to access token** On, ID/userinfo Off → Save.
3. **OpenHIM Console** → **Clients** → **+ Client** → **Client ID** `3130400221`, **Roles** add `np-fhir-clients`, **Authentication** leave with no password (JWT-only) → Save.

Fallback if the `sub` override is rejected

`onboard-hospital.sh` fails at step 4/5 with `token sub is '<uuid>', expected '<nhfr>'`. Leave the OpenHIM `clientID` as the Keycloak service-account UUID and add to `mediator/.env`:

```
FACILITY_REGISTRY_JSON={"<keycloak-service-account-uuid>":"3130400221"}
```

then `docker compose up -d --force-recreate mediator`. The mediator maps the UUID → NHFR code (`resolveFacility()` in `mediator/src/providers/jempiProvider.js`).

Verify (any option):

```
TOKEN=$(./keycloak/get-jwt-client-token.sh 3130400221)
echo "$TOKEN" | cut -c1-20     # non-empty
```

---

8. Smoke test — end to end

```
TOKEN=$(./keycloak/get-jwt-client-token.sh 3130400221)

curl -k -sS -X POST https://localhost:5000/fhir/Patient \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/fhir+json' \
  -d '{"resourceType":"Patient","name":[{"family":"SHRESTHA","given":["KAMANA"]}],
       "gender":"female","birthDate":"2001-06-25",
       "telecom":[{"system":"phone","value":"9800000000"}]}' | jq '{id, identifier, meta}'
```

Pass condition: `201`/`200` with an opaque UUID `id` and, in `identifier[]`, both `http://mohp.gov.np/fhir/NamingSystem/nepal-health-id` (11 digits) and `http://mohp.gov.np/fhir/NamingSystem/jempi-golden-uid`; and a `meta.tag` `source-facility` = `3130400221`.

- **GUI check:** OpenHIM Console → **Transactions** → the request → **Orchestrations** shows `JeMPI register` → `HAPI read Patient by Health ID` → `PUT HAPI Patient`.
- **Dedup check:** re-POST the identical body → the **same** `id` and identifiers, no new record.
- **Read-back:** `curl -k -sS https://localhost:5000/fhir/Patient/<id> -H "Authorization: Bearer $TOKEN" | jq .id`.

Postman helper: `node keycloak/token-server.mjs`, then `http://localhost:4099/token/3130400221` returns the token JSON.

---

9. FHIR Intake mediator (`POST /ingest`)

`POST /ingest` sits in front of the orchestrator:

Content-Type	Body	Action
`application/json`	one raw CDRS encounter row	validate → Patient + Encounter transaction Bundle → forward to `/fhir`
`application/fhir+json`	a supported resource or a `transaction`/`batch` Bundle	forward unmutated
anything else	—	`415`

The facility is the OpenHIM-set `X-OpenHIM-ClientID` (the caller's NHFR code) and nothing else; inbound `X-NHFR-ID` / `organization_id` are ignored. The mediator resolves `GET Organization?identifier=nhfr|<clientID>` (exactly one → `422` otherwise) and stamps `managingOrganization` / `serviceProvider`.

Set `OPENHIM_CHANNEL_ROLE=np-fhir-clients` in `fhir-intake-mediator/.env` to reuse the hospital clients from step 7, then `docker compose up -d --force-recreate fhir-intake-mediator`.

Smoke test:

```
TOKEN=$(./keycloak/get-jwt-client-token.sh 3130400221)
curl -k -sS -X POST https://localhost:5000/ingest \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"hospital_patient_number":"2405078","encounter_number":"SMOKE-1",
       "first_name":"CHANDRIKA","last_name":"KANDEL","sex":"Female",
       "department_type":"C","department_id":"22","department_name":"Family Medicine",
       "visit_date_ad_time":"2026-07-28 00:02:00.000"}'
```

`201`. A second POST of the same `encounter_number` must not create a duplicate Encounter.

Unit tests (no network, no credentials): `docker exec fhir-intake-mediator npm test`.

---

10. Duplicate-review mediator (`/admin/review/*`)

The `admin-mediator` finds near-miss duplicate patients that JeMPI's own deterministic `crRegister` never flags, scores them with the same m/u weights JeMPI is deployed with, and merges a confirmed pair in **both** JeMPI and HAPI.

The admin web UI is a separate project (`../../admin_portal`, **not in this repo**). To exercise the mediator directly you need its system client:

Option A — script:

```
docker exec -e ADMIN_CLIENT_PASSWORD='<same-as-BFF>' \
  admin-mediator node scripts/create-admin-client.js
```

Option C — GUI: OpenHIM Console → **Clients** → **+ Client** → ID `admin-portal`, role `np-admin-clients`, **Authentication** Basic, set the password.

Exercise it:

```
curl -sS -u admin-portal:<pw> -X POST  "https://localhost:5000/admin/review/scan" -k | jq
curl -sS -u admin-portal:<pw>          "https://localhost:5000/admin/review/tasks?status=OPEN"
        -k | jq '.counts'
```

---

11. Service management & reboot persistence

Action	Command
start	`docker compose up -d`
stop (safe)	`docker compose stop`
start after stop	`docker compose start`
restart one service	`docker compose restart <svc>` (does **not** reload `.env`)
reload `.env` for one service	`docker compose up -d --force-recreate <svc>`
status + health	`docker compose ps`
logs	`docker compose logs -f <svc>`
full reset (destructive)	`docker compose down -v --rmi all`

Every service has `restart: unless-stopped` and Docker is `systemctl enable`d, so the stack returns after a reboot. Verify: `systemctl is-enabled docker` → `enabled`; `sudo reboot`; then `docker compose ps` shows all `Up`.

---

12. Starting over (full reset) — destructive

```
docker compose down -v --rmi all
```

Drops: Keycloak H2 (realm + all clients/mappers), OpenHIM Mongo (clients, channels, transactions), HAPI Postgres (+ `admin_review`); forces an image re-pull. **Survives on disk:** `keycloak/keys/*.pem` (hospital keypairs — reused by `create-jwt-client.sh`), `*/.env` (but `OPENHIM_PASSWORD` is now wrong — root resets to `openhim-password`), `openhim-config/realm-public.pem` (**stale**), `hapi-config/`, `jempi-config/`. **JeMPI golden records are untouched** (JeMPI is external), so a "fresh" stack still matches previously registered patients.

Rebuild order: step 1 → step 3 (reset + propagate root password) → step 4 → step 5 → step 7 per hospital → step 8 to verify. Only if you also deleted `openhim-config/` do you need step 0d's `touch` again.

---

13. Quick reference

Task	Command
Create the `.env` files	`cp <svc>/.env.example <svc>/.env` (0d)
Pre-create the key file	`touch openhim-config/realm-public.pem` (0d)
Bring the stack up	`docker compose up -d`
Reload a changed `.env`	`docker compose up -d --force-recreate <svc>`
Create the realm	`./keycloak/setup.sh`
Create/update an Organization	`./keycloak/create-org.sh <nhfr> "<name>"`
Re-extract realm key	`./keycloak/extract-realm-pubkey.sh`
Onboard a hospital	`./keycloak/onboard-hospital.sh <nhfr> "<name>"`

Task	Command
Hospital token	`./keycloak/get-jwt-client-token.sh <nhfr>`
Postman token helper	`node keycloak/token-server.mjs (:4099)`
Check JeMPI	`curl <JEMPI_BASE_URL>/countRecords`

14. Troubleshooting

Symptom	Fix
Console calls fail, network error	trust the cert — https://localhost:8080/heartbeat (step 2)
Mediator missing under **Mediators**	`OPENHIM_PASSWORD` ≠ root password (step 3b)
`.env` change ignored	used `restart`, not `up -d --force-recreate`
`JWT could not be verified: secret or public key must be provided`	`realm-public.pem` empty or a directory — step 5 recovery
`JWT could not be verified` (other)	`realm-public.pem` stale — re-run step 5
401 at `:5000` with a fresh token	`iss` mismatch — token `iss` must equal `http://localhost:8083/realms/openhie`
403 at `:5000`	Client lacks the channel role — re-run step 7
404 at `:5000/...`	channel missing — `docker compose restart <mediator>`
`onboard-hospital.sh` fails at 4/5	Keycloak won't override `sub` — use the `FACILITY_REGISTRY_JSON` fallback (step 7)
`502` + `JeMPI register failed`	`JEMPI_BASE_URL` wrong / unreachable — check step 0e from host **and** `docker exec mediator wget -qO- $JEMPI_BASE_URL/countRecords`
container can't reach `10.8.0.1`, host can	masquerade the docker subnet out `wg0` (0e Option B)
Keycloak realm gone after a stop	`docker compose down` was used — use `stop`/`start`
`502` only from `admin-mediator`	it needs its **own** `JEMPI_BASE_URL` in `admin-mediator/.env`